PostgreSQL · RAG 운영 인수인계서

test_items2 원본·청크·임베딩·API·모델 교체 운영 가이드

기준일: **2026-10-19 (Asia/Seoul)** · 문서 v1.0 · API 0.3.0

대상: 다른 PC에서 이 DB와 RAG API를 사용해 Shopping Brain/Commerce 서비스를 운영할 개발자와 운영자.

이 문서는 실제 5432 서버의 메타데이터·현재 파일·실행 컨테이너를 읽기 전용으로 확인해 작성했다. 현재 상태, 향후 작업 절차, 미구현 기능을 구분한다. 문서 작성 중 데이터·계정 권한·활성 색인·모델·API 설정을 바꾸거나 검색 품질 테스트를 실행하지 않았다. 비밀번호·API 키·개인별 인증정보는 포함하지 않는다.

0. 가장 먼저 알아둘 사항

- DB 호스트는 chancelab-spark.tailbc019f.ts.net, PostgreSQL 포트는 **5432**다. 이 문서의 모든 DB 작업 대상은 이 서버다.
- 원본 상품·옵션·리뷰는 **DB toss → 스키마 test_items2 → products / product_variants / reviews**에 있다.
- 청크 본문과 임베딩 벡터는 **별도 DB toss_rag → 스키마 rag → evidence_chunks**에 있다.
- **test_items2는 toss에서는 스키마 이름이고, toss_rag에서는 검색 프로필 이름으로도 사용한다. test_items2라는 DB가 있는 것은 아니다.**
- API 기본 주소는 **https://chancelab-spark.tailbc019f.ts.net:8002**다. Tailscale의 8002는 HTTPS다.
- Test-NetConnection 성공은 TCP 연결만 확인한다. DB 로그인·권한과 API의 HTTP/JSON 결과는 별도로 확인한다.
- 현재 원본은 상품 1,500개, 옵션 1,917개, 리뷰 46,100개다. 상품과 옵션이 반드시 1:1인 데이터가 아니다.
- 현재 활성 색인은 114,106청크, 384차원이다. 다른 임베딩 모델로 바꾸면 같은 차원이어도 전체 재임베딩이 필요하다.

문서 구성

1. 전체 구조와 용어 — PDF 2쪽
2. Tailscale·DB·API 접속 — PDF 2쪽
3. 실제 DB·스키마·권한 — PDF 4쪽
4. 원본 테이블과 ID 관계 — PDF 5쪽
5. RAG 테이블·활성 프로필·검색 인덱스 — PDF 7쪽
6. 원본 파일·청킹 파일·전체 벡터 내보내기 위치 — PDF 11쪽
7. 확인용 SQL과 실제 조회 순서 — PDF 12쪽
8. RAG API와 호출 예시 — PDF 13쪽
9. 서비스·Shopping Brain 연동 기준 — PDF 14쪽
10. 실행 설정·재시작·배포·복구 — PDF 14쪽
11. 데이터 변경 시 반영 방식 — PDF 17쪽
12. 모델 종류와 현재 환경 — PDF 18쪽
13. 임베딩 모델 교체 절차 — PDF 18쪽
14. 활성화·되돌리기 및 차원 변경 — PDF 20쪽
15. LLM·재정렬 모델 변경 범위 — PDF 21쪽
16. 백업·복원·이관 — PDF 22쪽

17. 장애 대응과 인수 확인 항목 — PDF 23쪽

18. 현재 제한·검증 범위·전달 파일 — PDF 23쪽

부록 A. 요청·응답·오류·Node.js/Python 상세 API 계약 — PDF 24쪽 부록 B. 읽기 전용 SQL, 현재 DDL, 버전·모델 목록 파일 — PDF 33쪽

1. 전체 구조와 용어

한 PostgreSQL 서버 안에 여러 DB가 있고, 각 DB 안에 여러 스키마와 테이블이 있다. DB를 바꾸려면 새 연결을 열어야 한다. 문서에서 toss.test_items2.products라고 쓰는 것은 DB/스키마/테이블을 나타내는 표기다. toss에 연결한 실제 SQL에서는 FROM test_items2.products로 쓴다.

```
DGX Spark / PostgreSQL :5432 / 컨테이너 toss-postgres
≣≣ postgres                관리용 기본 DB
≣≣ toss                    Commerce 및 원본 DB
≣  ≣≣ public               Commerce 13테이블 + 보조 2테이블
≣  ≣≣ test_earphones       products / product_variants / reviews
≣  ≣≣ test_items           products / product_variants / reviews
≣  ≣≣ test_items2          products / product_variants / reviews
≣≣ toss_rag                검색 색인 DB
  ≣≣ rag
    ≣≣ index_profiles      프로필 → 현재 활성 run
    ≣≣ index_runs          색인 생성 이력·모델·버전·상태
    ≣≣ indexed_products    검색용 정규화 상품 스냅샷
    ≣≣ evidence_chunks     청크 본문·출처·384차원 벡터
    ≣≣ quarantined_products  격리 상품 기록

다른 PC → Tailscale → HTTPS :8002 → RAG API
                    ≣≣ 5432/toss_rag : 검색·근거
                    ≣≣ 5432/toss     : 옵션 가격·재고
Shopping Brain → RAG 후보 → Commerce 상품 연결 → 별도 LLM 답변
```

DB 간 외래키로 toss_rag와 toss를 직접 연결하는 구조는 아니다. product_id·source_dataset·source_table·dataset_version을 보존하고 서비스/ETL에서 두 DB의 연결을 확인한다. 한 DB 연결에서 다른 DB 테이블을 일반 JOIN으로 바로 조회할 수 있다고 가정하지 않는다.

2. Tailscale·DB·API 접속

2.1 다른 PC의 사전 조건

Tailscale에 로그인하고 DGX 접근이 허용된 네트워크 또는 공유 권한을 갖는다. 5432/8002에 대한 정책 허용 여부는 해당 PC에서 확인한다. 이름은 chancelab-spark.tailbc019f.ts.net, 현재 Tailscale IPv4는 100.114.218.1이다.

```
Test-NetConnection chancelab-spark.tailbc019f.ts.net -Port 5432
Test-NetConnection chancelab-spark.tailbc019f.ts.net -Port 8002
Invoke-RestMethod -Uri 'https://chancelab-spark.tailbc019f.ts.net:8002/healthz'
```

TcpTestSucceeded=True여도 비밀번호·DB 권한·TLS·HTTP 경로까지 통과한 것은 아니다. API 상태는 본문의 status=ok와 profile_name=test_items2를 함께 확인한다. 8002에 http로 접속하면 HTTPS 프록시에서 400이 나올 수 있다. HTTPS와 정확한 호스트명을 사용한다.

2.2 PostgreSQL 접속

클라이언트 항목	설정
Host	chancelab-spark.tailbc019f.ts.net
Port	5432
Database	원본 조회 toss / 청크 조회 toss_rag
User	관리자가 별도 전달한 계정. 서비스 계정 예: rag_runtime
Password	별도 보안 경로로 전달. 문서/소스/채팅에 기입하지 않음
SSL	현재 PostgreSQL ssl=on. 예시는 sslmode=require 사용

```
psql "host=chancelab-spark.tailbc019f.ts.net port=5432 dbname=toss user=rag_runtime sslmode=require" -W
psql "host=chancelab-spark.tailbc019f.ts.net port=5432 dbname=toss_rag user=rag_runtime sslmode=require" -W
```

위 명령은 psql이 설치된 PC에서 각각 실행한다. -W로 비밀번호를 입력하며 연결 문자열에 비밀번호를 쓰지 않는다. DBeaver/pgAdmin도 같은 항목을 사용한다. sslmode=require는 암호화를 요구하지만 CA·호스트명까지 검증하는 verify-full과 같지는 않다. verify-full을 쓰려면 관리자가 제공한 CA와 일치하는 인증서 호스트명 설정이 필요하다.

DB 로그인 후 SELECT current_database(), current_user, current_schema(), current_setting('search_path');로 접속 위치를 확인한다. public이 기본으로 보이더라도 최신 상품 조회 SQL에는 test_items2.products처럼 스키마를 명시한다.

pg_hba.conf는 계정·접속 IP·SSL 여부를 순서대로 판정한다. 일부 과거 관리자 계정에는 특정 IP만 허용하고 나머지를 reject하는 선행 규칙이 있으므로 TCP 접속 성공 후에도 로그인이 거부될 수 있다. 승인된 서비스 계정과 역할을 사용하고, 거부 시 관리자가 실제 클라이언트 주소와 적용 규칙을 확인한다.

2.3 API 접속 주소

용도	주소
Tailscale API 기본	https://chancelab-spark.tailbc019f.ts.net:8002
상태 확인	GET /healthz
상품·리뷰 근거 검색	POST /v1/rag/search
대체 NPM 도메인	https://rag.chanuk.theworkpc.com
DGX 내부	http://127.0.0.1:8002

현재 API 자체에는 별도 API 키 검증이 없다. Tailscale 경로는 해당 네트워크 접근 권한을 사용한다. NPM 도메인은 별도 공개 경로이므로 Tailscale 접근 제어와 동일한 것으로 간주하지 않는다. 인수 시 유지할 공개 범위와 인증 담당자를 정한다. 이 문서의 기본 연동 주소는 Tailscale이다.

루트 /에는 웹 화면이 없어 404가 나온다. /healthz로 상태를 확인한다. GET /v1/rag/search는 검색 메서드가 아니며 POST를 사용한다.

3. 실제 DB·스키마·권한

3.1 5432의 데이터베이스

DB	관측 크기	역할
postgres	7.5 MiB	관리용 기본 DB
toss	43.1 MiB	Commerce·원본
toss_rag	401.4 MiB	RAG 검색 색인

크기는 조사 시점의 pg_database_size이며 테이블/인덱스/기본 객체를 포함한다. 파일 ZIP 용량과 직접 동일하지 않다. API 연동은 toss와 toss_rag를 사용한다.

3.2 toss 내부 데이터

스키마	상품	옵션	리뷰
public	28	61	89
test_earphones	50	50	1,000
test_items	180	180	4,737
test_items2	1,500	1,917	46,100

public의 Commerce 테이블 13개는 products, product_variants, reviews, users, carts, cart_items, orders, order_items, payments, shipments, cancel_requests, action_requests, wishlist_items다. 보조 테이블은 _migrations, _scratch_owner다. public의 상품 28개가 test_items2의 상품 1,500개를 의미하는 것은 아니다.

현재 RAG의 새 데이터 원본은 test_items2다. test_earphones와 test_items는 기존 데이터로 보존한다. 기존 Commerce 코드가 FROM products로 조회한다면 기본 search_path에서 public.products를 읽을 수 있으므로 test_items2를 쓰려면 조회 경로·ID 매핑을 명확히 연결해야 한다.

3.3 현재 서비스 계정의 실제 권한

rag_runtime은 superuser가 아니다. toss.test_items2에서는 스키마 USAGE와 상품/옵션/리뷰 SELECT가 있다. toss.test_items에서는 상품/옵션 SELECT가 있고 리뷰 SELECT는 없다. public의 Commerce 상품과 test_earphones에는 별도 SELECT 권한이 확인되지 않았다.

중요: rag_runtime은 toss_rag.rag의 5개 테이블에서 SELECT뿐 아니라 INSERT/UPDATE 권한도 있으며 DELETE는 없다. API가 검색만 수행한다고 계정 전체가 읽기 전용인 것은 아니다. 운영에서 완전한 조회 전용 계정을 원하면 관리자가 별도 역할을 준비해야 한다. 기존 product_source_reader는 test_items2 조회 권한이 없으므로 이름만 보고 계정을 선택하지 않는다.

색인 생성·활성화·백업/복원은 승인된 운영 권한으로 수행한다. 비밀번호는 이 자료에 없으며, DB 접근 권한은 서버 셸·Docker·모델 변경 권한과 별도다.

4. 원본 테이블과 ID 관계

4.1 구조와 제약

products.id가 상품의 기본키다. product_variants.product_id는 products.id를 참조한다. reviews.product_id도 products.id를 참조한다. reviews.variant_id는 NULL 가능이며, 값이 있으면 (product_id, variant_id) 복합 FK가 같은 상품의 옵션을 참조한다. 가격·재고는 0 이상, 평점은 1~5, 리뷰 출처는 REAL/SYNTHETIC으로 제한된다.

현재 상품 1,500개에 옵션 1,917개가 있다. 신규 데이터가 복수 옵션을 포함하므로 상품=옵션 1개라는 가정을 서비스에 고정하지 않는다. reviews에만 있는 source_option_label은 원문 옵션 라벨이며 항상 구조화된 variant_id를 뜻하지 않는다.

4.2 test_items2.products

필드	타입 / NULL	의미
id	text / NOT NULL	상품 ID, PK
name	text / NOT NULL	원본 상품명 → 색인 title
brand	text / NOT NULL	브랜드
category	text / NOT NULL	beauty/fashion/food/living
description	text / NOT NULL	상품 설명
image_urls	jsonb / NOT NULL	이미지 URL 배열
specs_json	jsonb / NOT NULL	유형·분류·관측 사실·리뷰 표본 등의 원본 메타데이터
active	boolean / NOT NULL	활성 표시
version	integer / NOT NULL	원본 행 버전

4.3 test_items2.product_variants

필드	타입 / NULL	의미
id	text / NOT NULL	옵션 ID, PK
product_id	text / NOT NULL	상위 상품 FK
option_values_json	jsonb / NOT NULL	옵션 선택값
spec_overrides_json	jsonb / NOT NULL	옵션별 규격/속성
unit_price_won	integer / NOT NULL	정수 원화 옵션 단가
stock_qty	integer / NOT NULL	옵션 재고 수
active	boolean / NOT NULL	활성 표시
version	integer / NOT NULL	원본 행 버전

4.4 test_items2.reviews

필드	타입 / NULL	의미
id	text / NOT NULL	리뷰 ID, PK
product_id	text / NOT NULL	상위 상품 FK
variant_id	text / NULL 가능	특정 옵션 FK, NULL 허용
rating	integer / NOT NULL	1~5 정수 평점
body	text / NOT NULL	리뷰 본문
source_kind	text / NOT NULL	REAL 또는 SYNTHETIC
source_ref	text / NOT NULL	원본 출처 참조
written_at	timestamptz / NOT NULL	시간대 포함 작성 시각
source_option_label	text / NULL 가능	원문 옵션 라벨, NULL 허용

4.5 데이터 구성과 출처

카테고리	상품	리뷰
beauty	781	34,545
fashion	269	2,761
food	182	5,682
living	268	3,112

리뷰 출처	리뷰 수	variant_id가 NULL
REAL	41,523	0
SYNTHETIC	4,577	938

REAL은 원본에서 수집된 리뷰, SYNTHETIC은 합성 리뷰다. CANDIDATE 데이터라는 상태와 출처를 유지한다. 원본 가격/재고는 DB에 저장된 시연·관측 값이며 실시간 외부 쇼핑몰 재고를 뜻하지 않는다. 구조적 FK가 맞는 것과 상품 분류·리뷰 의미·추천 품질이 맞는 것은 별도로 확인해야 한다.

5. RAG 테이블·활성 프로필·검색 인덱스

5.1 현재 활성 색인

프로필	원본 범위	상품 / 청크
all_products	이전 test_items · beauty	180 / 6,741
test_items2	test_items2 · 4개 카테고리	1,500 / 114,106

```
profile_name = test_items2
active_run_id = e925490f-4f34-4c03-9248-eb9ebdfcde48
dataset_version = test_items2:v6.1-review-refined:sha256=94681a28271ca9b1:products=1500:reviews=46100
embedding_model = sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
embedding dimension = 384
chunker_version = source-evidence-token-v2:max_tokens=96
category_policy_version = source-declared-v6.1-v1
```

all_products의 활성 run은 49537fff-9b05-4b9c-aed1-c4e690a4f689이며 이전 뷰티 180개/6,741청크다. 이름과 달리 최신 전체 상품을 뜻하지 않는다. RAG API의 기본 프로필은 test_items2이고, 두 프로필은 분리되어 있다.

프로필은 검색 범위의 별칭, run은 특정 데이터·모델·청커로 생성한 색인 실행본이다. 항상 index_profiles.active_run_id로 활성 run을 구한 다음 해당 run의 상품·청크를 읽는다. evidence_chunks 전체를 run 조건 없이 조회하면 향후 과거/신규 색인이 섞일 수 있다.

5.2 테이블 역할과 핵심 필드

rag.index_profiles

필드	DB 타입	의미
profile_name	text	프로필 PK
active_run_id	uuid	활성 run FK
updated_at	timestamp with time zone	전환 시각

rag.index_runs

필드	DB 타입	의미
run_id	uuid	실행본 PK
dataset_version	text	원본 스냅샷/가공 버전
embedding_model	text	벡터 생성 모델 이름
chunker_version	text	청커 버전·예산
category_policy_version	text	분류 정책 버전
status	text	building/active/superseded/failed
counts	jsonb	상품·리뷰·청크 집계
error_code	text	실패 코드, NULL 가능
started_at	timestamp with time zone	시작 시각
completed_at	timestamp with time zone	완료 시각, NULL 가능
profile_name	text	소속 프로필

rag.indexed_products

필드	DB 타입	의미
run_id	uuid	색인 실행본
dataset_version	text	데이터 버전
category	text	대분류
product_id	text	원본 상품 ID
product_payload	jsonb	정규화 상품/옵션 스냅샷 JSON
price_amount	numeric	색인 시점 가격 스냅샷
price_currency	text	가격 통화

rag.evidence_chunks

필드	DB 타입	의미
run_id	uuid	색인 실행본
chunk_key	text	run+chunk 기반 전역 키, BM25 키
chunk_id	text	청크 ID
dataset_version	text	데이터 버전
category	text	대분류
product_id	text	원본 상품 ID
source_type	text	product 또는 review
source_id	text	원본 상품/리뷰 출처 ID
source_field	text	정규화 문서 필드 경로
start_offset	integer	근거 시작 문자 위치
end_offset	integer	근거 끝 위치, 미포함
evidence_text	text	인용할 청크 본문
embedding_text	text	실제로 임베딩한 문자열
source_metadata	jsonb	출처·옵션·평점·합성 표식
embedding	vector(384)	384차원 벡터
embedding_model	text	벡터 생성 모델
chunker_version	text	이 청크의 청커 버전

rag.quarantined_products

필드	DB 타입	의미
run_id	uuid	실행본
dataset_version	text	데이터 버전
category	text	대분류
product_id	text	격리 상품 ID
reason_code	text	격리 이유
policy_version	text	검사 정책
product_title	text	저장 메타데이터
main_category	text	저장 메타데이터
categories	jsonb	저장 메타데이터
type_signals	jsonb	유형 판정 근거
review_status	text	검수 상태
created_at	timestamp with time zone	격리 생성 시각

현재 활성 test_items2는 상품·옵션 청크 37,919개와 리뷰 청크 76,187개로 총 114,106개다. 벡터도 모두 저장되어 있다. source_type=product에는 옵션에서 만든 근거도 포함되므로 별도 variant source_type을 가정하지 않는다.

product_payload는 name을 title로, specs_json을 features로 매핑한 정규화 스냅샷이다. 원본 데이터를 완전히 대체하는 정본은 아니다. source_metadata에는 source_database=toss, source_dataset=test_items2, source_table, source_kind, variant_id, source_ref, synthetic 등의 출처를 보존한다.

5.3 DB 확장과 검색 방식

PostgreSQL 18.6, pgvector 0.8.7, pg_search 0.26.0을 사용한다. BM25는 별도 LLM 프로세스가 아니라 PostgreSQL의 pg_search/ParadeDB 인덱스와 검색 연산자를 사용한다.

현재 evidence_chunks_search_idx는 USING paradedb (chunk_key, run_id, embedding_text)로 구성된다. 벡터는 embedding <=> query_vector의 코사인 거리 순으로 조회한다. 관측한 rag 인덱스에는 HNSW/IVFFlat 인덱스가 없다. 따라서 "벡터 DB를 쓴다"는 말만으로 근사 벡터 인덱스가 있다고 판단하지 않는다. 데이터가 커질 때 실제 실행 계획과 지연을 측정해 별도 인덱스 설계를 검토한다. pgvector 공식 문서

6. 원본 파일·청킹 파일·전체 벡터 위치

6.1 서버의 실제 파일

대상	서버 경로
원본 JSON 디렉터리	/home/aiteam/projects/toss-ads-project/data/test_items2/source/shopping_brain_v6_1_common_dataset/data/
원본 파일	위 디렉터리의 products.json / variants.json / reviews.json
적재·색인 보고서	/home/aiteam/projects/toss-ads-project/data/test_items2/load_report.json / index_report.json / run.json
구조 검증·기존 탐색 평가	/home/aiteam/projects/toss-ads-project/data/test_items2/schema_validation_2026-10-19.json / search_quality_2026-10-19.md
원본+청크+전체 벡터 ZIP	/home/aiteam/projects/toss-ads-project/data/exports/test_items2_chunked_full_2026-10-19.zip
ZIP 체크섬	위 ZIP 경로에 .sha256을 덧붙인 파일
모델 캐시	/home/aiteam/.cache/codex-rag-fastembed
DB 저장소 bind mount	/home/aiteam/projects/toss-ads-project/data/product-rag/postgres
컨테이너 내부 PGDATA	/var/lib/postgresql/18/docker

실시간 검색은 폴더의 JSONL을 매번 읽는 것이 아니라 toss_rag.rag.evidence_chunks를 조회한다. 청킹된 텍스트와 전체 벡터의 파일 형태는 아래 ZIP 내부에 있다. ZIP의 JSONL은 내보내기 시점 스냅샷이며 DB 수정이 자동 반영되지 않는다. 모델 캐시는 가중치·토크나이저 파일이며 상품별 임베딩 벡터 저장소가 아니다.

6.2 전체 데이터 ZIP 내부

```
test_items2_chunked_full_2026-10-19.zip
▤▤ shopping_brain_v6_1_common_dataset/data/
▤  ▤▤ products.json
▤  ▤▤ variants.json
▤  ▤▤ reviews.json
▤▤ database_exports/toss/test_items2/
▤  ▤▤ products.jsonl
▤  ▤▤ product_variants.jsonl
▤  ▤▤ reviews.jsonl
▤  ▤▤ schema.sql
▤▤ database_exports/toss_rag/rag/
▤  ▤▤ evidence_chunks.jsonl       본문·출처·전체 384차원 벡터
▤  ▤▤ indexed_products.jsonl
▤  ▤▤ index_runs.jsonl
▤  ▤▤ index_profiles.jsonl
▤  ▤▤ quarantined_products.jsonl
▤  ▤▤ schema.sql
▤▤ metadata/                 내보내기·적재·색인 보고서
▤▤ README_KO.md
▤▤ SHA256SUMS.txt
```

ZIP 크기는 **273,558,559 bytes (273.56 MB / 260.89 MiB)**이고 파일 19개를 포함한다. evidence_chunks.jsonl의 압축 해제 크기는 713,276,797 bytes다. JSON 벡터를 포함하므로 DB의 벡터 바이너리 크기와 다르다.

이번 문서 작성 중 다시 계산한 ZIP SHA-256:

```
5cffff3e0a656cbbdf55a0b76c7493a0373c48a77d5d6770af9a65865e66b808
```

ZIP은 원본/색인 데이터 전달 자료다. 계정 비밀번호, 모델 캐시, Docker 이미지, 서버·NPM 설정 전체를 포함하는 일괄 복구 패키지가 아니다. 이 인수인계 ZIP에도 273 MB 데이터 ZIP 자체를 중복 포함하지 않으며, 위치·체크섬·구성 목록으로 전달한다.

7. 확인용 SQL과 실제 조회 순서

부록 B의 SQL 파일은 읽기 전용이다. 명시된 DB에 연결해 실행한다. 코드 블록의 SQL은 운영자가 실행할 예시이며 이 문서 작성 중 모든 내용의 결과를 재검증한 것은 아니다.

7.1 원본 확인 — toss에 연결

```
BEGIN READ ONLY;
SELECT current_database(), current_user;
SELECT id, name, category, specs_json->>'product_type' AS product_type
FROM test_items2.products
WHERE name ILIKE '%니플%'
ORDER BY name;
SELECT id, product_id, unit_price_won, stock_qty
FROM test_items2.product_variants
WHERE product_id = 'p-v4-o-0286';
SELECT id, product_id, variant_id, source_kind, rating, body
FROM test_items2.reviews
WHERE product_id = 'p-v4-o-0286'
ORDER BY written_at DESC, id LIMIT 10;
ROLLBACK;
```

위 실제 상품 ID는 "[초슬림 실리콘] 다슈 맨즈 에어핏 니플 밴드 52매(단품/더블기획)"의 ID다. 상품 존재 예시이지 남성용 적합성·추천 순위를 검증한 결과는 아니다.

7.2 활성 청크 확인 — toss_rag에 연결

```
BEGIN READ ONLY;
SELECT p.profile_name, r.run_id, r.dataset_version,
     r.embedding_model, r.chunker_version, r.status, r.counts
FROM rag.index_profiles p
JOIN rag.index_runs r
  ON r.profile_name=p.profile_name AND r.run_id=p.active_run_id
WHERE p.profile_name='test_items2';

SELECT c.product_id, c.source_type, c.source_id, c.source_field,
     c.evidence_text, c.source_metadata,
     vector_dims(c.embedding) AS dimensions
FROM rag.evidence_chunks c
WHERE c.run_id=(SELECT active_run_id FROM rag.index_profiles
          WHERE profile_name='test_items2')
  AND c.product_id='p-v4-o-0286'
ORDER BY c.source_type,c.source_id,c.start_offset LIMIT 10;
ROLLBACK;
```

전체 벡터가 필요 없는 점검에서는 embedding 전체 대신 vector_dims만 읽는다. 상품 ID는 toss.test_items2.products.id, review의 source_id는 toss.test_items2.reviews.id와 대조한다. offset은 정규화 문서 기준이므로 화면 인용에는 evidence_text를 사용한다.

두 DB의 조회를 한 SQL에 섞지 않는다. 클라이언트의 연결 두 개 또는 서비스의 두 DB 연결을 사용한다. 청크가 있다는 사실만으로 해당 상품이 최종 API 결과에 포함됐다고 판단하지 않는다. 프로필·카테고리·유형·Commerce·근거 선택을 통과해야 한다.

8. RAG API와 호출 예시

8.1 기본 요청

Method: POST · URL: https://chancelab-spark.tailbc019f.ts.net:8002/v1/rag/search · Content-Type: application/json

```
{
  "request_id": "handover-example-001",
  "profile": "test_items2",
  "original_question": "니플패치 찾아줘. 리뷰도 보여줘",
  "query": "니플패치",
  "category": "fashion",
  "filters": {},
  "required_conditions": [],
  "preferences": [],
  "top_k": 5
}
```

original_question과 query는 필수다. profile 생략 시 test_items2, category 생략 시 그 프로필의 전체 카테고리를 검색한다. product_type은 합의된 유형을 보낼 때만 사용한다. 니플패치의 독립 유형은 현재 사전에 없으므로 위 예시는 product_type을 생략한다.

8.2 Windows PowerShell 호출

```
$baseUrl = 'https://chancelab-spark.tailbc019f.ts.net:8002'
$payload = @{
  request_id = 'powershell-handover-001'
  profile = 'test_items2'
  original_question = '니플패치 찾아줘. 리뷰도 보여줘'
  query = '니플패치'
  category = 'fashion'
  top_k = 5
} | ConvertTo-Json -Depth 10
$result = Invoke-RestMethod -Uri "$baseUrl/v1/rag/search" `
  -Method Post -ContentType 'application/json; charset=utf-8' `
  -Body ([System.Text.Encoding]::UTF8.GetBytes($payload)) -TimeoutSec 30
$result | ConvertTo-Json -Depth 20
```

예시는 사용법이며 이번 문서에서 검색 POST를 실행한 결과가 아니다. 30초는 예시의 클라이언트 제한으로 운영 SLA가 아니다. Postman은 포함된 rag-api-tailscale.postman_collection.json을 Import한다. 기본 base_url과 profile은 이미 Tailscale/test_items2로 설정했다.

8.3 응답을 처리할 때

- HTTP 상태와 JSON 본문의 status/error_code를 함께 확인한다. 프록시 오류는 HTML일 수 있다.
- status=ok이면 items를 사용하고, no_results/low_relevance이면 search_context.reason을 확인한다. HTTP 200만으로 상품이 있다고 판단하지 않는다.
- profile_name=test_items2, dataset_version, ranking_version을 기록해 원본·색인·코드 버전을 구분한다.
- product_metadata에 이미지 URL과 전체 옵션이 포함되지는 않는다. Commerce 원본에서 보완한다.
- commerce.variant_id는 선택 옵션이고 evidence.source_metadata.variant_id는 리뷰의 출처 옵션이다. NULL 리뷰를 특정 옵션 후기처럼 표시하지 않는다.
- REAL/SYNTHETIC, CANDIDATE, demo, is_live_market_price 상태를 보존한다.

필드별 타입·길이·기본값·오류 코드·상세 JSON·Node.js/Python 코드는 부록 A에 모두 수록했다.

9. 서비스·Shopping Brain 연동 기준

원문 → Brain의 목적/상품군 해석 → RAG 검색 → 원본 상품/옵션 연결 → 근거 확인 → 별도 LLM 답변 → 카드 표시 순서로 운영한다. 이 API는 범용 카테고리 분류 LLM이 아니다. 자연어를 입력할 수 있지만 원문의 핵심 상품이 query에 보존되도록 Brain이 해석한다.

실제 Brain이 remote API를 쓰는지 local JSONL 검색을 쓰는지 먼저 확인한다. 현재 서버의 API를 수정해도 다른 저장소의 local RAG가 자동으로 바뀌지는 않는다. 설정 키 이름은 실제 Brain 실행본의 계약을 따른다.

최신 범위는 profile=test_items2이며 beauty 기본값을 모든 요청에 강제하지 않는다. "니플패치"를 beauty로 보내면 fashion 데이터가 빠진다. 카테고리가 확실하지 않으면 생략한다. 상품명 지정 검색의 관련성·일치 정보를 후속 가격/리뷰 재정렬에서도 유지한다.

current required_conditions/preferences의 임의 문자열은 검증된 필터가 아니다. unverified_conditions로 표시한다. 성별 적합성·통기성·효능을 근거 없이 조건 충족으로 표현하지 않는다. RRF는 검색 순위 점수이며 성능·효능·만족도 점수가 아니다.

리뷰 어댑터는 REAL과 variant_id=NULL을 수용해야 한다. SYNTHETIC 전용 어댑터를 그대로 쓰면 이번 데이터의 REAL 41,523개가 탈락할 수 있다. product_id·source_id·데이터 버전·인용 원문을 보존한다.

LLM 답변 호출이 실패해도 이미 받은 상품 카드가 남을 수 있다. 후보 수신 성공과 답변 생성 실패를 분리해 "상품 후보는 찾았지만 추천 이유와 리뷰 설명을 생성하지 못했어요"처럼 안내한다.

10. 실행 설정·재시작·배포·복구

다음 명령은 DGX 셸과 Docker 권한이 있는 운영자가 실행한다. 다른 PC에서 DB/API에 접근할 수 있다는 것만으로 서버 관리 권한이 생기지는 않는다.

10.1 현재 실행 자산

항목	현재 값
PostgreSQL 컨테이너	toss-postgres
PostgreSQL 이미지	toss-ads-postgres-rag:18.6-vector-0.8.7-search-0.26.0
API 컨테이너	product-rag-api
API 이미지	toss-ads-product-rag-service:24a1051-test-items2
API 실행	python -m toss_ads.rag serve --host 127.0.0.1 --port 8002
API 네트워크	Docker host network
API 재시작 정책	unless-stopped
적용 소스	24a1051fff84bb4cf5441e90824e9e9be20e18ec
코드 브랜치	feature/rag-recommendation-quality
소스 저장소	https://github.com/hongbinwee/codyssey-final-toss-backend
현재 worktree	/home/aiteam/projects/toss-ads-project/codyssey-final-toss-backend/.worktrees/rag-recommendation-quality
컨테이너 내 코드	/app/src/toss_ads/rag
환경 파일	/home/aiteam/.local/state/toss-rag-api/test-items2-bbb66a2dc206/runtime.env
실행 관측 기록	동일한 상태 디렉터리의 activation-report.json
이전 API 보관	product-rag-api-before-test-items2-bbb66a2d

환경 파일과 previous-container.json에는 인증정보가 들어 있을 수 있어 접근 제한된 서버 경로에서만 관리한다. 인수인계 ZIP에는 포함하지 않는다. 실행 코드는 이미지에 복사되어 있으며 개발 worktree의 소스를 수정해도 API가 즉시 바뀌지 않는다.

10.2 비밀값을 제외한 주요 환경변수

변수	현재 값 / 역할
FASTEMBED_CACHE_PATH	/home/aiteam/.cache/codex-rag-fastembed
RAG_EMBEDDING_MODEL	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
RAG_CHANNEL_DEPTH	20
RAG_RRF_K	60
RAG_EMBEDDING_BATCH_SIZE	64
RAG_DEFAULT_PROFILE	test_items2
RAG_ALLOWED_PROFILES	test_items2,all_products
RAG_REQUIRE_IN_STOCK	true
RAG_CANDIDATE_DEPTH	50
RAG_CHUNKS_PER_PRODUCT	3
RAG_CANDIDATE_SCAN_LIMIT	2000
COMMERCE_SCHEMA_MAP	test_items2:test_items2,test_items:test_items,test_earphones:test_earphones,commerce:public

RAG_DATABASE_URL은 127.0.0.1:5432/toss_rag, COMMERCE_DATABASE_URL은 127.0.0.1:5432/toss를 가리킨다. Docker host network이므로 여기서 loopback은 Spark 호스트의 DB를 뜻한다. DB 비밀번호는 별도 환경 파일에서 관리한다. RAG_HTTP_TIMEOUT_SECONDS는 설정 이름만 보고 전체 검색/LLM의 강제 타임아웃으로 해석하지 않는다. 실제 호출·프록시별 제한을 확인한다.

10.3 기본 점검과 재시작

```
docker ps --filter name=product-rag-api --filter name=toss-postgres
docker logs --tail 100 product-rag-api
docker logs --tail 100 toss-postgres
curl http://127.0.0.1:8002/healthz
tailscale serve status --json
systemctl is-active toss-rag-lan.socket toss-rag-lan.service

docker restart product-rag-api
curl http://127.0.0.1:8002/healthz
```

단순 API 재시작에는 DB 재시작이 필요하지 않다. docker restart는 기존 컨테이너의 환경변수를 사용한다. 환경 파일을 수정했더라도 새 값이 반영되려면 컨테이너를 재생성해야 한다. 모델 이름만 바꾸고 재시작하는 작업은 색인 교체 절차가 아니다.

10.4 Tailscale와 NPM의 경로

Tailscale Serve는 chancelab-spark.tailbc019f.ts.net:8002 HTTPS를 http://127.0.0.1:8002에 전달한다. NPM 도메인은 http://172.30.1.53:8002로 전달한다. LAN 수신은 /etc/systemd/system/toss-rag-lan.socket와 toss-rag-lan.service가 172.30.1.53:8002 → 127.0.0.1:8002로 중계한다.

예전 502는 API가 loopback에서만 수신하여 NPM의 LAN 연결이 거부된 것이 원인이었다. 현재 Tailscale·LAN·NPM /healthz는 정상 응답을 확인했다. LAN 주소가 바뀌면 socket unit과 NPM 목적지를 같이 갱신한다. NPM 설정만 바꾸고 API가 LAN에서

받는지 확인하지 않으면 같은 문제가 재발할 수 있다.

기존에 요청받아 실행한 npm exec localtunnel --port 8002 --local-host 127.0.0.1 프로세스도 조사 시점에 실행 중이었다. 이 경로는 임시 공유용이며 인수인계 기본 주소로 사용하지 않는다. Tailscale 주소를 사용한다고 NPM/임시 공유 경로가 자동으로 중단되는 것은 아니다. 운영 소유자는 유지할 경로와 인증 범위를 함께 인수한다. 이번 문서 작업에서는 어떤 경로도 중지하거나 변경하지 않았다.

10.5 코드 배포

Backend의 feature/rag-* 브랜치에서 수정하고 PR/검토 후 사용자 지시에 따라 main을 반영한다. main 직접 커밋·푸시는 하지 않는다. 현재 운영 소스는 이미 푸시된 24a1051 개발 브랜치이고 main 병합과 운영 적용은 서로 다른 상태다. 신규 설치 시 main만 받아 현재 RAG 코드가 있다고 가정하지 않는다.

```
git clone --branch feature/rag-recommendation-quality https://github.com/hongbinwee/codyssey-final-toss-backend.git
cd codyssey-final-toss-backend
git rev-parse HEAD
docker build -f deploy/product-rag.Dockerfile -t toss-ads-product-rag-service:NEW_TAG .
```

NEW_TAG는 운영자가 정할 새 이미지 태그다. requirements-runtime-snapshot.txt는 현재 설치 버전의 관측 목록이며 재현성 참고 자료다. 저장소 requirements는 범위 지정이므로 다시 설치하면 버전이 달라질 수 있다. 검토된 버전·ARM64 이미지·모델 캐시를 묶어 배포한다.

기존 이미지를 보관한 채 승인된 환경 파일과 같은 host network/캐시 mount/127.0.0.1:8002 실행 옵션으로 새 컨테이너를 만든다. 같은 주소에 두 API를 동시에 기동하지 않는다. /healthz의 프로필·모델 호환 상태와 실제 연동 요청을 확인한다. 이 문서는 새 이미지 빌드/배포를 실행한 결과가 아니다.

이전 보관 컨테이너는 이전 beauty 180개 기본 API다. 이를 되살리면 서비스 프로필 범위도 예전으로 돌아가므로 최신 test_items2 복구와 동일하게 취급하지 않는다. 현재 API의 설정/이미지를 별도 보관해 모델 교체의 정확한 복구 대상을 준비한다.

11. 데이터 변경 시 반영 방식

- 신규 상품·이름·설명·스펙·리뷰 변경은 toss.test_items2 원본과 RAG 스냅샷/청크를 함께 갱신하는 ETL·재색인이 필요하다. DB에 INSERT했다고 검색에 자동 노출되지 않는다.
- 현재 API는 가격·재고를 toss의 옵션 행에서 조회하므로 해당 값은 검색 시 DB 조회에 반영된다. 색인 payload의 가격 스냅샷과 구분한다.
- 원본 category나 상품 ID를 바꾸면 색인·Commerce 연결이 어긋날 수 있다. 기존 ID를 임의로 재발급하지 않는다.
- 현재 Commerce 조회 SQL은 active 컬럼을 필터하지 않는다. active=false만 설정해서 API 비노출을 보장하지 않는다. 운영 비노출 정책을 구현하고 색인/조회 경로를 함께 맞춰야 한다.
- 현재 상품 카탈로그 캐시는 run/category별 약 60초이고, 캐시 내용 자체가 indexed_products의 스냅샷이다. 원본 수정이 캐시 만료만으로 자동 재색인되는 것은 아니다.
- API는 전체 상품에 대한 임의 조건을 모두 검증하지 않는다. 성별·통기성·민감성·효능 등은 별도의 데이터 계약과 근거 검증이 필요하다.

초기 적재/색인은 /tmp/toss_test_items2_worker.py와 task_chunker를 이용한 별도 작업이었다. 현재 서버에 참고 파일이 남아 있지만 /tmp는 영구 운영 소스가 아니다. 해당 worker는 이미 test_items2 프로필이 있으면 중단하는 가드와 완료 즉시 활성화하는 흐름이 있다. 재실행용 일반 CLI로 사용하지 않는다. 재색인 작업은 다음 장의 단계와 현재 저장소 API를 바탕으로 별도 운영 진입점을 준비해야 한다.

12. 모델 종류와 현재 환경

구분	역할	변경 시 주요 영향
임베딩 모델	문서·질문 → 검색 벡터	전체 문서 재임베딩, 필요 시 재청킹·차원 변경
BM25	DB의 키워드 검색 인덱스	텍스트/토크나이저/인덱스 설정 변경 시 재구성 검토
재정렬 모델	후보 순위를 다시 평가	현재 전용 모델 미구현. 추가 서비스/코드·평가 필요
답변 생성 LLM	검색 근거로 자연어 답변 생성	Brain/LLM 제공자 설정 변경. 보통 RAG 벡터 재생성 불필요

12.1 현재 임베딩 실행 환경

모델: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2, 출력 384차원, 모델 입력 상한 128토큰, 기존 청크 예산 96토큰. 실제 가중치 소스는 Qdrant/paraphrase-multilingual-MiniLM-L12-v2-onnx-Q의 model_optimized.onnx다.

현재 Python/OS 아키텍처는 aarch64이며 FastEmbed 0.9.0, ONNX Runtime 1.31.0, langchain-community 0.3.31, langchain-core 0.3.86이다. ONNX 사용 가능 provider는 AzureExecutionProvider와 CPUExecutionProvider이며 CUDA provider는 없다. 현재 임베딩을 GB10 GPU에서 실행 중이라고 해석하지 않는다.

관측한 ONNX 캐시 snapshot은 faf4aa4225822f3bc6376869cb1164e8e3feedd0이고 파일 크기는 235,052,644 bytes다. 파일 SHA-256과 경로는 model-cache-manifest.json에 있다. 이 관측값이 코드에서 revision을 고정한다는 뜻은 아니다. 현재 설정은 모델 이름을 기록하며 별도 model revision 고정 설정은 없다.

FastEmbed는 지원되는 ONNX 모델과 추가 모델 등록 경로를 제공하지만, 특정 Hugging Face 모델 이름을 환경변수에 넣는 것만으로 모두 실행되는 것은 아니다. 모델 변경은 설치된 버전의 지원 목록·ONNX 파일·입력 형식·출력 차원·실행 provider와 함께 확인한다. FastEmbed 공식 문서

12.2 지원 모델 목록 확인 — 다운로드나 모델 교체 없음

```
docker exec product-rag-api python -c 'from fastembed import TextEmbedding; import json;
print(json.dumps(TextEmbedding.list_supported_models(), ensure_ascii=False, indent=2, default=str))'
docker exec product-rag-api python -c 'import onnxruntime; print(onnxruntime.get_available_providers())'
```

이 목록은 해당 FastEmbed 버전이 등록한 모델 목록이다. 현재 런타임에서 목록 41개를 확인해 fastembed-supported-models-runtime.json으로 보관했다. 목록에 있다고 ARM64/GB10 GPU 실행과 한국어 추천 품질까지 검증된 것은 아니다. GPU로 바꾸려면 호환되는 ONNX Runtime/CUDA/ARM64 구성을 별도로 준비하고 실제 provider 사용을 확인한다.

13. 임베딩 모델 교체 절차

13.1 단순 설정 변경으로 끝나지 않는 이유

현재 DB 컬럼은 vector(384), embeddings.py는 EMBEDDING_DIMENSION=384와 MODEL_MAX_TOKENS=128, store.py의 쓰기/검색과 retriever.py의 질의 벡터 검사도 384로 고정돼 있다. 768/1024차원 모델을 RAG_EMBEDDING_MODEL에 넣으면 코드 검증이나 DB 저장에서 실패한다.

같은 384차원 모델이라도 벡터 공간은 다를 수 있으므로 예전 문서 벡터와 새 질문 벡터를 섞으면 안 된다. 문서와 질문은 같은 모델·revision·토크나이저·정규화·입력 prefix 정책을 사용해야 한다. 현재 API는 활성 run의 embedding_model과 실행 설정을 비교해 다르면 configuration_error를 반환한다. 이 비교는 모델 이름 기준이므로 같은 이름의 가중치 revision 변경까지 감지하지는 못한

다. 파일 해시·revision·입력 규약을 별도 manifest로 관리한다.

현재 API 프로세스는 임베딩 모델 하나만 로딩한다. 다른 모델로 교체한 뒤 이전 모델의 all_products를 허용해 두면 그 프로필 검색은 모델 불일치 오류가 날 수 있다. 새 모델로 각 프로필을 재색인하거나, 지원 범위를 test_items2로 한정하는 등의 프로필 정책을 전환 전에 정한다.

13.2 모델 교체 전 준비

1. 현재 API 이미지·환경 파일·모델 캐시·active_run_id·원본/색인 백업을 보관한다. 새로운 모델이 기존 캐시를 대체하거나 예전 run을 삭제하지 않게 한다.
2. 후보 모델의 한국어/상품명/동의어 성능, ONNX/ARM64 지원, 출력 차원, 입력 길이, query/document prefix와 pooling/normalization, 라이선스를 기록한다.
3. 원본 스냅샷과 검수한 평가 질문/정답을 고정한다. 세제·파우치/파우더·니플패치·캡슐로션·드레스의 상품명/모델번호·조건 불일치·목록 밖 사례를 포함한다.
4. 메모리·CPU·처리량을 확인하고 batch_size를 정한다. 검증 전에는 GPU 설치나 큰 배치가 더 빠를 것이라고 가정하지 않는다.
5. 모델 이름뿐 아니라 가중치 revision/파일 해시와 어댑터 설정을 고정해 재현할 수 있게 한다.

13.3 문서/질문 입력 형식 확인

설치된 LangChain FastEmbed 어댑터의 embed_query는 model.query_embed를 사용한다. embed_documents는 doc_embed_type='passage'일 때 passage_embed, 기본값 'default'일 때 embed를 사용한다. 현재 create_embedder는 doc_embed_type을 별도로 지정하지 않는다.

새 모델이 passage/query prefix를 요구한다면 어댑터의 해당 경로를 모델 규약에 맞게 설정해야 한다. 문자열에 prefix를 붙이는 처리와 라이브러리의 자동 처리를 중복 적용하지 않는다. 새 모델의 공식 입력 규약과 실제 생성 문자열을 기록한다.

13.4 청크를 재사용할 수 있는 경우

기존 evidence_chunks의 evidence_text/embedding_text·출처·offset을 새 run으로 복사하면서 벡터만 새 모델로 계산하는 방식을 고려할 수 있다. 단, 모든 embedding_text를 새 토크나이저로 다시 세어 새 모델의 입력 예산에 들어가는지 먼저 확인해야 한다. 길이 초과를 조용히 truncate해서 색인하지 않는다.

현재 chunker.py의 EMBEDDING_MODEL도 상수이며 chunk_id 해시 구성에 모델 이름이 들어간다. 새 모델로 재생성할 때 모델 메타데이터만 바꾸지 말고 모델 인자를 청커/ID 생성 경로까지 전달하도록 수정한다. 기존 embedding_text를 재사용하더라도 새 run 청크의 ID·모델·데이터 버전 규약을 일관되게 만든다. 기존 run의 ID와 행은 그대로 보존한다.

토크나이저·입력 prefix·문서 구성·청킹 정책이 달라져 예산을 넘거나 근거 구성이 바뀌면 원본으로 돌아가 재청킹한다. 새 청크의 source_field/offset/evidence_text와 원본 관계를 다시 확인한다. 현재 배포 소스의 청커는 v3지만 활성 색인은 v2다. 새 청커를 사용할 때는 청커 버전과 실제 생성 결과를 따로 기록한다.

13.5 새 색인 생성 — 운영자/개발자가 구현할 단계

현재 배포 CLI에는 serve만 있으며 --reindex/--model 같은 완성된 모델 교체 명령은 없다. 아래는 저장소의 실제 RagStore API를 사용하는 작업 순서이며 그대로 붙여 실행할 완성 스크립트가 아니다. 기존 임시 worker를 수정 없이 실행하는 명령도 제공하지 않는다.

```
1. 승인된 색인 쓰기 계정과 새 모델 설정을 별도로 준비한다.
2. Settings / create_embedder 또는 모델별 어댑터를 구성한다.
3. RagStore.begin_index_run(..., profile_name="test_items2")로
   같은 프로필에 새 building run을 만든다. 기존 active는 유지한다.
4. upsert_products(new_run, IndexedProduct 목록)로 상품 스냅샷을 저장한다.
5. 원본에서 새 청크를 만들거나 검증된 기존 청크를 읽는다.
   EvidenceChunk의 dataset_version / embedding_model / chunker_version을
   실제 생성 기준으로 기록한다. 원본 ID·출처·리뷰 표식은 보존한다.
6. embed_documents([...])로 새 벡터를 배치 생성한다.
7. 길이·유한값·개수·모델 정보 확인 후
   upsert_chunks(new_run, [(EvidenceChunk, vector), ...])로 저장한다.
8. 아직 activate_dataset을 호출하지 않는다.
9. 새 run의 무결성과 검색 품질을 검증하고 결과를 남긴다.
10. 승인된 전환 시점에만 새 run을 활성화하고 API 모델을 맞춘다.
```

색인 계정에는 해당 rag 테이블 INSERT/UPDATE와 원본 SELECT가 필요하다. runtime 계정에 현재 쓰기 권한이 있다는 사실과 실제 운영 작업의 승인/책임은 별도다. 다른 프로필에 만든 run을 test_items2의 active_run_id에 바로 넣는 것은 복합 FK/소속 계약과 맞지 않는다.

13.6 새 색인의 검증 항목

- 상품·리뷰·청크 집계, ID/FK/원본 출처, REAL/SYNTHETIC·NULL 옵션 표식을 확인한다.
- vector_dims·NaN/Inf·모델 이름/revision·입력 길이를 확인한다. 새 청킹이면 청크 수는 기존 114,106개와 달라질 수 있다.
- 원본 표본을 포함해 source_field의 [start_offset, end_offset)와 evidence_text가 맞는지 확인한다.
- 같은 고정 정답에서 BM25, 새 벡터, 결합 검색을 비교한다. Hit@1/5, MRR/nDCG, 무관한 상품 노출, 정상 질문 거절, 예산/재고 위반, p95·메모리·처리 시간을 함께 기록한다.
- 새 run이 building이면 일반 API는 active run만 보므로 직접 run_id를 지정한 RagStore.search_bm25/search_vector와 새 embedder로 오프라인 검증한다. 임시 외부 포트나 새 DB를 만들 필요는 없다.
- 모델 비교 오프라인 결과와 실제 Brain→Commerce→LLM→카드 E2E를 구분한다. 검증 결과가 없으면 교체 효과를 확정하지 않는다.

위 검증은 교체 담당자가 수행할 절차이며 이번 문서 작성에서 실행하지 않았다.

14. 활성화·되돌리기 및 차원 변경

14.1 같은 차원의 새 모델로 전환

현재의 단일 API에서 모델과 active run을 무중단으로 동시에 바꾸는 자동화는 없다. 짧은 점검 시간에 API 요청을 멈추고 전환하는 방식을 기본으로 계획한다.

1. 검증 완료한 새 building run과 새 모델 이미지·환경 파일을 준비한다. 현재 API 환경/이미지와 이전 active run은 보관한다.
2. API 요청을 중지하거나 기존 컨테이너를 중지한다.
3. 같은 test_items2 프로필의 새 run에 대해 RagStore.activate_dataset(new_run, counts, profile_name='test_items2')을 호출한다. 이 메서드는 기존 active를 superseded로 바꾸고 새 building을 active로 바꾸며 프로필 포인터를 갱신한다.
4. 새 API의 RAG_EMBEDDING_MODEL, 지원 프로필, 입력 어댑터와 모델 캐시를 새 run과 맞춘다. 환경변수 변경은 컨테이너 재생성이 필요하다.
5. 같은 기존 8002에서 API를 시작한다. healthz가 새 데이터·프로필·모델 호환 상태인지 확인하고 실제 검색·Brain E2E를 확인한 뒤 요청을 재개한다.
6. 이전 run/이미지/캐시는 복구 기간 동안 보존한다. 5432 원본은 삭제하거나 재발급하지 않는다.

activate_dataset은 building run을 대상으로 하며, 이미 superseded인 예전 run을 그대로 다시 활성화하는 만능 rollback 함수가 아니다. 전환 중 API 모델만 먼저 바꾸거나 색인 포인터만 먼저 바꿔 요청을 받으면 configuration_error 또는 잘못된 벡터 비교가 발생할 수 있다.

14.2 되돌리기 절차

API 요청을 멈춘 뒤 이전 모델 이미지/환경 파일을 준비한다. DB 관리 트랜잭션에서 test_items2의 advisory lock과 프로필 행 잠금을 획득하고, 현재 active가 기록한 새 run과 일치하는지 확인한다. 되돌릴 run이 같은 프로필 소속이며 원본/모델/청커 정보를 보존한 superseded run인지 확인한다.

그 트랜잭션 안에서 현재 active를 superseded로 변경하고, 대상 이전 run을 active로 변경한 다음 index_profiles.active_run_id와 updated_at을 갱신한다. 각 UPDATE 행 수가 기대와 맞지 않으면 ROLLBACK한다. 전체 성공 시 COMMIT하고 이전 모델 API를 기동한다. healthz와 실제 검색을 확인한 뒤 요청을 재개한다.

단순히 active_run_id 한 컬럼만 바꾸면 index_runs.status와 어긋나 API가 활성 run을 찾지 못할 수 있다. 프로필당 active 하나라는 unique index와 (profile_name, active_run_id) FK도 지켜야 한다. 모델 전환/rollback SQL은 담당자가 현재 run ID와 권한을 확인한 뒤 작성·검토하고 실행한다. 이 문서에는 즉시 실행되는 DB 변경 SQL을 넣지 않았다.

14.3 384가 아닌 차원의 모델

현재 vector(384) 컬럼에는 다른 길이의 벡터를 저장할 수 없다. 출력 차원이 다르면 기존 운영 컬럼을 즉석에서 덮어쓰지 않고 새 벡터 테이블/버전 구조 등 보존 가능한 마이그레이션을 설계해야 한다. 같은 5432의 toss_rag 안에서 분리 구조를 만들 수 있지만 현재 코드에는 테이블별 모델 라우팅이 구현되어 있지 않다.

변경 지점	필요 작업
src/toss_ads/rag/embeddings.py	EMBEDDING_DIMENSION, 모델 입력 길이, query/document 어댑터
src/toss_ads/rag/chunker.py	고정 EMBEDDING_MODEL, 모델을 포함한 chunk_id 생성·메타데이터·토큰 예산
src/toss_ads/rag/store.py	upsert_chunks/search_vector 길이 검사, 읽기·쓰기 대상 및 필요 인덱스
src/toss_ads/rag/retriever.py	질의 벡터의 길이 검사, 모델/색인 선택 계약
DB DDL/마이그레이션	새 vector(N) 컬럼/테이블, 제약·인덱스·기존 run 보존
색인 worker/정규화	새 토크나이저, chunk ID/출처/버전, 배치 저장·재시작
설정·API 계약	모델과 프로필의 호환 범위, health/readiness, 오류 처리
운영	자원·지연·품질 비교, 전환·복구, 패키지/모델 캐시 보관

같은 테이블에 모델 이름만 달리 기록한다고 서로 다른 차원을 저장할 수 있는 것은 아니다. 차원을 잘라 맞추거나 0을 채워 넣는 방법으로 모델을 호환시키지 않는다. 공식적으로 지원되는 차원 축소 방식이 있더라도 문서와 질의에 같은 처리를 적용해 새 색인을 만들고 검증해야 한다.

15. LLM·재정렬 모델 변경 범위

15.1 답변 생성 LLM 변경

RAG_EMBEDDING_MODEL은 검색 모델이며 최종 답변 LLM 설정이 아니다. 답변 LLM을 바꾸려면 실제 Shopping Brain 실행본의 provider·endpoint·model 이름·인증정보·컨텍스트 길이·출력 토큰·타임아웃·동시성·구조화 응답 계약을 확인한다. 현재 외부 Brain의 정확한 배포본/제공자 설정은 이 문서에서 확인하지 못했으므로 임의의 환경변수 이름이나 모델 주소를 확정하지 않는다.

LLM만 바꾸고 상품 문서/임베딩 규약이 같다면 일반적으로 벡터를 다시 만들 필요는 없다. 다만 query 재작성·카테고리 판단·리뷰 인용·JSON 형식·모델 오류 처리와 실제 화면 E2E는 다시 검증한다. 새 모델이 빈 claims/잘못된 ID/원문에 없는 인용·효능을 생성하지 않는지 확인한다.

15.2 재정렬 모델 추가/교체

현재는 별도 Cross-Encoder/재정렬 모델을 호출하지 않는다. 어휘 일치 우선순위와 상품 단위 RRF를 사용한다. 재정렬 모델을 추가하려면 후보 입력 구성, 모델 어댑터, CPU/GPU 자원, 배치·타임아웃·실패 시 정책, 점수 기록과 평가가 필요하다. 단순 환경변수 변경으로 활성화되는 기능은 아니다.

재정렬 모델만 바꾸는 경우 기존 임베딩을 재사용할 수 있지만, 상품 관련성·명시 조건·상품명 우선순위를 점수로 무시하지 않게 한다. 후보 회수율이 낮은 문제는 재정렬만으로 해결되지 않을 수 있다.

16. 백업·복원·이관

16.1 백업 대상

- DB toss의 원본 및 필요한 Commerce 데이터, DB toss_rag의 색인·활성 프로필·run 메타데이터.
- API 이미지/소스 커밋, PostgreSQL 확장 버전/이미지, 실제 설치 패키지 목록.
- 모델 가중치·토크나이저·캐시 manifest, 청커/분류표/입력 규약.
- 비밀값을 제외한 실행 설정과 별도 보안 경로의 자격증명, NPM/Tailscale 경로, LAN unit.

실행 중인 PostgreSQL의 데이터 디렉터리를 일반 파일 복사하는 것만으로 일관된 백업을 만들었다고 판단하지 않는다. pg_dump 등 논리 백업 또는 검증된 물리 백업 절차를 사용한다. toss와 toss_rag는 DB가 달라 각각의 pg_dump를 실행하면 스냅샷 시점이 다를 수 있다. 서로 연결되는 원본/색인이 변경되지 않는 점검 구간이나 일관된 버전 기록을 사용한다.

16.2 DGX에서 논리 백업 예시 — 운영자가 실행

```
mkdir -p /home/aiteam/backups/rag-handover

docker exec toss-postgres sh -lc 'pg_dump -Fc --no-owner --no-privileges -U "$POSTGRES_USER" -d toss' >
/home/aiteam/backups/rag-handover/toss.dump

docker exec toss-postgres sh -lc 'pg_dump -Fc --no-owner --no-privileges -U "$POSTGRES_USER" -d toss_rag' >
/home/aiteam/backups/rag-handover/toss_rag.dump

docker image save -o /home/aiteam/backups/rag-handover/rag-api-image.tar toss-ads-product-rag-service:24a1051-test-items2
```

명령 종료 코드·생성 크기·체크섬을 확인한다. 위 명령은 이번 문서 작성 중 실행하지 않았다. 계정/비밀번호/권한은 --no-owner/--no-privileges 덤프에 포함되지 않으므로 별도 복구 계획이 필요하다. 공개 API에서 DB 백업 파일을 내려받는 기능은 없다.

16.3 복원 순서

승인된 복원 검증 환경에서 PostgreSQL 18.6과 호환되는 vector/pg_search 확장을 준비한다. 새 DB에는 확장을 먼저 설치하고 원본/색인 DDL과 데이터를 복원한다. 복원 대상의 기존 운영 데이터를 무조건 --clean으로 덮어쓰지 않는다.

원본 ID와 NULL·REAL/SYNTHETIC·시간대·JSON을 유지한다. rag 쪽에서는 index_runs/상품/청크/격리 기록과 index_profiles의 FK 의존성을 지켜 복원한다. 제공 ZIP의 JSONL은 INSERT 가능한 SQL이나 pg_restore용 custom dump가 아니며, vector 배열을 pgvector 값으로 매핑할 import 작업이 필요하다. model 이름/차원·active 상태·출처 연결을 확인한 다음 API를 기동한다.

한 번의 pg_dump/ZIP 생성만으로 복원 검증이 끝난 것은 아니다. 실제 복원 결과의 계정 권한·확장·상품/리뷰 ID·활성 run·API 검색과 Brain 화면을 확인한다. 기존 5432 원본/색인과 기존 ZIP은 검증이 끝날 때까지 보존한다.

17. 장애 대응과 인수 확인 항목

증상	확인 순서
5432 TCP 실패	Tailscale 로그인/ACL·호스트명·서버/포트 수신·컨테이너·방화벽
5432 TCP 성공, 로그인 실패	DB명·계정·SSL·비밀번호·pg_hba 선행 reject·실제 클라이언트 IP
permission denied for schema/table	해당 DB의 USAGE/SELECT 등 역할 확인. 이름만으로 읽기 권한 추정 금지
8002 TCP 성공, HTTP 400	Tailscale 경로에 HTTPS와 정확한 호스트명을 사용했는지 확인
루트 / 404	정상 API 동작일 수 있음. /healthz 사용
NPM만 502	172.30.1.53:8002 수신, LAN socket/service, NPM upstream 확인
healthz error/not_ready	기본 프로필 active run·모델 일치·DB 권한/연결 확인
unsupported_category/profile	test_items2와 지원 카테고리를 확인. all_products=이전 beauty
상품이 원본에 있는데 검색 누락	활성 run·원본/색인 ID·카테고리·유형·Commerce·근거 선택·실제 Brain 경로 확인
카드가 있는데 설명 실패	RAG 응답과 별도 LLM 호출 failure.code/httpStatus를 구분
모델 변경 후 configuration_error	API 실행 모델과 요청 프로필 active run의 embedding_model 일치 여부
신규 데이터가 검색에 안 보임	원본 수정 후 새 색인 생성·활성화가 있었는지 확인

인수 담당자가 확인할 순서

1. Tailscale 접근 권한과 5432/8002의 TCP를 확인한다.
2. toss/toss_rag 로그인과 필요한 SELECT 권한을 확인한다. 비밀번호는 별도로 전달받는다.
3. 원본 스키마·상품/옵션/리뷰 수와 현재 active_run_id를 확인한다.
4. Tailscale /healthz의 status/profile/dataset/categories를 확인한다.
5. Postman/클라이언트로 정상 상품·리뷰·가격/재고·빈 결과·입력 오류 요청을 실행하고 실제 결과를 저장한다.
6. 실제 Brain이 이 remote API와 test_items2 원본을 사용하는지 확인하고 카드·답변까지 점검한다.
7. 서버 셸/Docker·운영 환경 파일·모델 캐시·백업·복원·권한 담당자를 인수한다.
8. 모델 변경·색인 전환·복구 담당자와 점검 시간을 합의하고 이전 run/이미지 보존 기간을 정한다.

이 목록은 수신 운영자가 수행할 인수 절차이며 이번 문서의 테스트 통과 기록이 아니다.

18. 현재 제한·검증 범위·전달 파일

현재 확인: 실제 5432 DB·테이블·컬럼·제약·권한·확장·활성 run·집계, ZIP 경로/내부 파일 목록/체크섬, 실행 컨테이너·설정·패키지·provider·캐시 파일, Tailscale/NPM /healthz의 HTTP 200과 test_items2.

미확인/미실행: 수신 PC의 실제 DB 인증·API 호출, 최신 검색 품질 평가·외부 Brain E2E, 모델 교체·재색인·활성화·rollback·복원. 기존 탐색 평가 숫자는 과거 데이터/코드의 제한된 결과이므로 이 문서에서 새 코드의 성과로 보고하지 않는다.

알고 있어야 할 제한: 임의 조건/선호 미검증, 상품 유형 사전의 미등록·오분류, 소재와 상품 유형의 혼동, 완전한 핵심명/모델번호 후보 경로와 Brain 순위 보존의 미완성, 현재 active=false 비노출 미구현, 외부 Brain 어댑터/배포 동일성 미확인, 별도 재정렬 모델·범용 모델 교체 CLI 부재. 운영 인수는 연결 상태와 추천 품질을 나누어 판단한다.

제공 파일

- 이 인수인계 Markdown 및 같은 내용의 공유 PDF.
- observed-inventory.json: 비밀값을 제외한 실제 메타데이터/설정/집계.
- toss.test_items2.schema.sql, toss_rag.rag.schema.sql: 현재 schema-only DDL. 참고/복원 설계용이며 자동 실행하지 않는다.
- sql/: DB를 구분한 읽기 전용 점검 SQL.
- rag-api-tailscale.postman_collection.json 및 examples/: 호출용 자료. 자동 테스트 스크립트는 없다.
- rag-search.request.schema.json: 현재 실행 요청 모델.
- requirements-runtime-snapshot.txt, fastembed-supported-models-runtime.json, model-cache-manifest.json.
- deploy/systemd/: 적용된 LAN 수신 unit의 원본.
- data-archive-locator.json: 대용량 원본+벡터 ZIP 위치·크기·체크섬·내부 파일 목록.

문서·운영 참고 파일은 Backend worktree에 로컬 저장했다. 이번 작업에서는 새 커밋·GitHub 푸시·main 병합·제품 코드 변경·운영 설정 변경을 수행하지 않았다. 실행 중인 소스는 기존 24a1051이며 이전 네트워크 unit 적용과 문서의 Git 반영 상태는 별도로 관리한다.

부록 A. 상세 API 계약과 클라이언트 코드

이 부록의 기본 주소는 Tailscale HTTPS :8002다. 가상 응답 값과 사용 예시는 실제 검색 테스트 결과가 아니다. 서버 코드 24a1051/API 0.3.0을 기준으로 한다.

A-1. 요청 필드

Content-Type: application/json · UTF-8 · 요청 본문 최대 64 KiB.

필드	타입 / 기본값	규칙
original_question	string, 필수	사용자 원문. 1~4,000자, 공백만 불가
query	string, 필수	검색에 쓸 핵심어/질문. 1~1,000자, 공백만 불가
request_id	string 또는 null / null	추적용 ID, 최대 128자. 자동 생성·중복 제거 키가 아님
profile	string 또는 null / 서버 기본값	현재 기본 test_items2. all_products도 허용
category	string 또는 null / 전체	프로필 내부 검색 범위. 1~128자
product_type	string 또는 null / 자동 추론	등록된 유형 ID 또는 지원되는 카탈로그 유형. 1~128자
filters	object / {}	아래 가격·재고 조건. null 불가
required_conditions	string[] / []	최대 20개, 각 1~500자. 임의 조건은 미검증으로 반환
preferences	string[] / []	최대 20개, 각 1~500자. 임의 선호는 미검증으로 반환

필드	타입 / 기본값	규칙
top_k	integer / 10	1~20. 실제 반환 수는 더 적을 수 있음

미등록 요청 필드는 422 invalid_request다. 원문과 query는 모두 필요하며, 원문에 있는 상품을 재작성 과정에서 다른 상품군으로 바꾸지 않는다. 숫자·문자열 타입을 임의로 바꾸거나 top_k에 문자열을 보내지 않는다.

자연어 문자열을 넣을 수 있지만 API가 범용 LLM 의도 분석이나 대화 기억을 수행하는 구조는 아니다. Brain에서 목적·카테고리를 판단하고 짧은 상품 핵심어를 query로 전달하는 방식을 권장한다. 예: original_question은 "운동 후 먹을 단백질 보충용 상품 추천", query는 "단백질 보충식".

A-1.1 프로필과 카테고리

profile	실제 데이터 범위	지원 category
test_items2	새 통합 데이터 1,500상품	beauty, fashion, food, living
all_products	이전 test_items 180상품	beauty

프로필 이름 all_products가 전체 최신 상품을 뜻하지 않는다. 현재는 이전 뷰티 색인이다. 새 데이터는 test_items2를 명시하거나 기본값을 사용한다. category 생략 시 선택한 프로필의 전체 카테고리에서 검색한다. 니플패치를 beauty로 제한하면 fashion 상품은 제외된다. 프로필에 없는 category는 unsupported_category다.

A-1.2 가격·재고 필터

filters 내부 필드	타입 / 기본값	적용
price_min	number 또는 null / null	0 이상, 옵션 단가 하한 포함
price_max	number 또는 null / null	0 이상, 옵션 단가 상한 포함
currency	string 또는 null / null	가격 범위 지정 시 필수. 현재 KRW만 지원
in_stock	boolean 또는 null / 서버 기본값	현재 생략/null은 true. false는 재고 0인 옵션도 허용

```
{
  "price_min": 10000,
  "price_max": 20000,
  "currency": "KRW",
  "in_stock": true
}
```

price_min ≤ price_max여야 한다. 가격과 재고는 **같은 옵션**에서 확인하며 조건에 맞는 최저 단가 옵션을 고른다. 배송비·수량·쿠폰·총 결제액 필터는 구현되어 있지 않다. 예산 총액을 옵션 단가와 동일하게 취급하지 않는다.

required_conditions에 "통기성 좋음"을 넣어도 해당 조건을 만족하는 상품만 보장해서 거르는 것은 아니다. 현재는 unverified_conditions로 반환하며 별도 근거 확인이 필요하다.

product_type은 합의한 유형만 보낸다. 예: detergent, protein_food, pouch. 니플패치는 현재 별도 등록 유형이 없으므로 예시처럼 query를 보내고 product_type은 생략한다. 이 필드에 임의 문장을 넣는 것은 범용 분류 요청이 아니다.

A-2. 응답 구조와 상태

검색 응답에서 먼저 HTTP 상태를 확인하고, 이어 본문 status와 error_code를 처리한다.

본문 status	일반 HTTP	처리
ok	200	items 사용. 조건 검증·출처·Commerce 상태도 함께 확인
no_results	200	적격 후보 없음. search_context.reason 확인
low_relevance	200	유형 충돌 또는 카탈로그 정체성 일치 부족. 확인 질문 등으로 처리
error	422/500/503/504	error_code에 따라 입력 수정 또는 서버 확인

A-2.1 검색 응답 예시

아래는 **형식 설명용 가상 값**이며 실제 검색 실행 결과가 아니다. 전체 예시는 examples/rag-search.response.example.json에 있다.


```
{
  "request_id": "other-pc-test-001",
  "status": "ok",
  "profile_name": "test_items2",
  "dataset_version": "<현재 색인 버전>",
  "applied_filters": {"category": "fashion", "in_stock": true},
  "supported_categories": ["beauty", "fashion", "food", "living"],
  "unverified_conditions": [],
  "items": [{
    "product_id": "example-product-001",
    "category": "fashion",
    "product_metadata": {"title": "설명용 상품명"},
    "rrf_score": 0.03,
    "evidence": [{"source_type": "review", "source_id": "example-review-001", "evidence_text": "설명용 인용문입니다."}],
    "commerce": {"variant_id": "example-variant-001", "validation_status": "verified_database_row"},
    "ranking": {"relevance_qualified": true, "title_identity_match": 1, "lexical_coverage": 1.0, "requested_product_type": null}
  }],
  "error_code": null,
  "search_context": {"ranking_version": "product-hybrid-v2", "relevance_gate": "product_identity_v2"}
}
```


위 예시는 필드를 일부 생략한 발췌다. 실제 요청 결과의 근거 출처와 데이터 버전을 확인하고, 실제 인용에는 반환된 evidence_text를 사용한다.

A-2.2 공통 필드

필드	의미
request_id	요청 추적값. 오류 처리 과정에서 일부 문자가 정리될 수 있음
profile_name	실제 선택한 프로필
dataset_version	실제 활성 색인 버전. 실패 단계에 따라 null
applied_filters	적용한 카테고리·유형·가격·재고 조건
supported_categories	선택한 프로필의 카테고리 목록
unverified_conditions	적용을 보장하지 못하는 필수/선호 조건
items	상품 단위 결과 배열
error_code	성공/빈 결과이면 null, 오류이면 코드
search_context	검색 버전·유형 추론·후보 수·빈 결과 이유 등 진단 정보

입력 검증 실패나 예외로 반환하는 오류 응답은 profile_name/search_context가 없을 수 있다. 클라이언트는 이 필드의 항상 존재를 가정하지 않는다. search_context의 세부 키는 처리 단계에 따라 달라진다.

A-3. 상품·옵션·리뷰 근거의 의미

결과 필드	의미와 사용법
items[].product_id	Commerce 원본 상품과 연결할 ID. 프로필/출처와 함께 사용
category	반환 상품의 실제 대분류
product_metadata	title, brand, main_category, price, average_rating, rating_number, source_dataset, dataset_release_status 중 저장된 필드
commerce	DB에서 조회한 선택 옵션·단가·재고·시각·출처 상태
rrf_score	검색 순위 결합 점수. 정답 확률·효능·만족도 점수 아님
ranking.title_identity_match	전체 정규화 제목 일치 2 / 제목 토큰 일치 1 / 제목 토큰 일치 없음 0
ranking.lexical_coverage	제목·브랜드·유형에 대한 어휘 일치 진단값
ranking.relevance_qualified	현재 구현의 관련성 검사를 통과했다는 값. 사용자 조건 전체 충족 보증 아님
evidence	상품당 최대 3개의 선택 근거 청크

검색 응답 product_metadata에는 이미지 URL·전체 옵션 목록이 포함되지 않는다. 카드 이미지·상세 정보는 product_id와 source_dataset을 이용해 해당 Commerce 데이터에서 가져온다. 전체 평점과 리뷰 수는 저장된 경우에만 포함되며, 검색된 일부 리뷰의 평균으로 전체 평점을 덮어쓰지 않는다.

A-3.1 commerce

commerce에는 variant_id, unit_price_won, currency, stock_qty, checked_at, validation_status, source_database, source_schema, price_basis, is_simulated, purchase_mode, is_live_market_price, dataset_release_status, source_price_evidence_status가 들어갈 수 있다.

현재 test_items2는 purchase_mode=demo, is_live_market_price=false다. verified_database_row는 DB 행과 가격·재고를 확인했다는 뜻이며 실제 시장 재고·실시간 판매 가격을 보장하지 않는다. is_simulated는 원본 표식을 전달하므로 boolean 또는 null일 수 있다. stock_qty와 unit_price_won도 시연 데이터라는 상태를 유지해서 표시한다.

A-3.2 evidence

근거 필드	의미
product_id / chunk_id	상품 ID / 청크 ID. 청크 ID는 리뷰 ID가 아님
source_type / source_id	review이면 원본 리뷰 출처 ID. 상품 근거이면 상품 출처 ID
source_field	색인에 사용한 정규화 문서의 필드 경로. 예: /text
source_version	해당 활성 데이터셋 버전
evidence_text	응답에서 실제 인용할 수 있는 근거 문자열
source_metadata	source_kind, synthetic, variant_id, rating, source_table, source_dataset, source_ref 등. 일부 키는 없거나 null
start_offset / end_offset	정규화 원문 필드 내 0부터 시작하는 문자 구간 [start, end)
bm25_rank / vector_rank	해당 검색 채널 내 청크 순위. 미포함 시 null
review_bm25_rank / review_vector_rank	리뷰 의도 시 추가되는 리뷰 전용 채널의 순위. null 가능
rrf_rank	반환 상품의 순서. 리뷰 평점이나 근거 신뢰도 점수 아님

review의 source_kind=REAL/SYNTHETIC을 보존하고 합성 리뷰를 실제 구매 경험으로 표시하지 않는다. source_metadata.variant_id=null인 상품 단위 리뷰도 허용한다. 이를 선택된 commerce.variant_id에 대한 특정 옵션 리뷰로 바꾸어 표현하지 않는다.

A-3.3 ID 연결 계약

- test_items2 결과의 product_id는 toss.test_items2.products.id와 연결한다.
- test_items2의 review 근거 source_id는 toss.test_items2.reviews.id와 연결한다. source_table/source_dataset도 확인한다.
- commerce.variant_id는 선택된 원본 product_variants.id다. 리뷰 근거의 variant_id와 의미가 다르다.
- 다른 프로필의 같은 제목 상품은 다른 ID를 가질 수 있다. 제목으로 프로필 간 ID를 대체하지 않는다.
- source_field와 offset은 정규화 문서 기준이다. PostgreSQL body 문자열에 무조건 직접 적용하지 말고 전달된 evidence_text를 사용한다.
- 기존 /v1/catalog/test-items/{product_id}/reviews는 별도 legacy 응답으로 ID를 해시 변환한다. test_items2 리뷰 API로 사용하지 않는다.

A-4. 오류와 빈 결과 진단

A-4.1 검색 API 오류 코드

error_code	HTTP	의미 / 대응
invalid_request	422	필수 필드·타입·길이·미등록 필드·가격 범위·64 KiB 제한 확인
unsupported_profile	422	test_items2 또는 all_products 사용
unsupported_category	422	선택 프로필의 supported_categories 확인
unsupported_currency	422	현재 가격 필터는 KRW만 사용
index_not_ready	503	해당 프로필 활성 색인 확인
configuration_error	503	실행 모델과 색인의 모델 등 서버 설정 확인
commerce_not_configured	503	Commerce 연결 구성 필요
commerce_source_unmapped	503	원본 데이터셋과 Commerce 스키마 매핑 확인
embedding_failed	503	쿼리 임베딩 생성 실패. 서버 로그 확인
database_failed	503	DB 연결·권한·SQL 실행 상태 확인
timeout	504	API 내부 TimeoutError 처리. 프록시 자체 504와 구분
search_failed	500	검색 처리 예외. request_id와 서버 로그 대조

```
{
  "request_id": "other-pc-test-001",
  "status": "error",
  "dataset_version": null,
  "applied_filters": {},
  "supported_categories": [],
  "unverified_conditions": [],
  "items": [],
  "error_code": "invalid_request"
}
```

위 오류 JSON도 형식 예시다. 프록시 자체 오류는 JSON 대신 HTML을 반환할 수 있다. HTTP 오류 여부를 먼저 확인하고 JSON Content-Type/파싱 실패도 처리한다. 422는 같은 요청을 그대로 반복하지 말고 입력을 수정한다.

A-4.2 HTTP 200이지만 검색 결과가 없는 경우

search_context.reason	의미
conflicting_or_ambiguous_product_type	원문·재작성 검색어·명시 유형 충돌 또는 모호함
category_type_mismatch	지정 카테고리와 추론 유형의 범위 불일치
no_matching_product_type	해당 유형의 후보 없음
no_catalog_identity_match	제목·브랜드·유형의 어휘 연결 부족
no_commerce_eligible_products	가격·재고 등 Commerce 조건 충족 후보 없음
no_eligible_candidates	검색·관련성·근거 선택 후 최종 후보 없음

A-4.3 접속 문제

증상	확인할 내용
도메인 / 가 404	웹 화면이 없는 정상 상태. /healthz 사용
GET /v1/rag/search 가 405	검색은 POST. 브라우저 주소창 입력은 GET임
도메인에서 502	NPM이 upstream에 연결할 수 있는지 확인. 목적지는 http://172.30.1.53:8002
내부 health는 정상, 도메인은 502	LAN 수신/네트워크/NPM 경로 확인. 과거에는 127.0.0.1 전용 바인딩이 원인이었음
브라우저 JS에서만 실패	FastAPI 앱에 별도 CORS 허용 미들웨어가 없음. 현재 NPM의 CORS 설정은 미검증. 서버 측 호출 또는 같은 출처 프록시 사용
상품 카드는 있지만 AI 답변 실패	이 API의 상품 응답과 별도 Brain/LLM 설명 생성을 각각 확인
잘못된 상품이 노출	profile·category·query·유형·관련성 검사와 Brain의 후속 재정렬 확인

정상 통신이 검색 품질을 보장하지 않는다. 이미 존재하는 상품도 잘못된 프로필/카테고리·상품 유형·후속 어댑터에서 누락될 수 있다.

A-5. Node.js와 Python 호출 예시

예시는 서버 측 실행용이며 자동 테스트 스크립트가 아니다. 타임아웃 30초는 호출 예시의 선택값으로, 운영 SLA나 API 강제 제한을 뜻하지 않는다.

A-5.1 Node.js — 내장 fetch를 제공하는 실행 환경

```
const response = await fetch(
 'https://chancelab-spark.tailbc019f.ts.net:8002/v1/rag/search',
 {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  signal: AbortSignal.timeout(30_000),
  body: JSON.stringify({
   request_id: 'node-example-001',
   profile: 'test_items2',
   original_question: '니플패치 찾아줘. 리뷰도 보여줘',
   query: '니플패치',
   category: 'fashion',
   top_k: 5
  })
 }
);
const text = await response.text();
let data;
try { data = JSON.parse(text); }
catch { throw new Error(`RAG proxy/non-JSON response: ${response.status}`); }
if (!response.ok || data.status === 'error') {
 throw new Error(`RAG ${response.status}: ${data.error_code ?? 'unknown'}`);
}
if (data.status !== 'ok') {
 console.log('검색 결과 없음:', data.search_context?.reason);
} else {
 console.log(data.items);
}
```

예를 들어 위 코드를 rag-search.mjs에 저장해 node rag-search.mjs로 실행할 수 있다. fetch의 네트워크 오류와 TimeoutError도 상위 호출부에서 처리한다.

A-5.2 Python — 표준 라이브러리

```
import json
import urllib.request
import urllib.error

payload = {
  'request_id': 'python-example-001',
  'profile': 'test_items2',
  'original_question': '니플패치 찾아줘. 리뷰도 보여줘',
  'query': '니플패치',
  'category': 'fashion',
  'top_k': 5,
}
request = urllib.request.Request(
  'https://chancelab-spark.tailbc019f.ts.net:8002/v1/rag/search',
  data=json.dumps(payload, ensure_ascii=False).encode('utf-8'),
  headers={'Content-Type': 'application/json'},
  method='POST',
)
try:
  with urllib.request.urlopen(request, timeout=30) as response:
    data = json.load(response)
  if data['status'] == 'error':
    print('RAG 오류:', data.get('error_code'))
  elif data['status'] != 'ok':
    print('검색 결과 없음:', data.get('search_context', {}).get('reason'))
  else:
    print(json.dumps(data, ensure_ascii=False, indent=2))
except urllib.error.HTTPError as error:
  print('HTTP 오류:', error.code, error.read().decode('utf-8', errors='replace'))
except (urllib.error.URLError, TimeoutError, json.JSONDecodeError) as error:
  print('연결·시간 초과·응답 형식 오류:', error)
```

A-6. Shopping Brain 연동 기준

1. 사용자 원문은 original_question에 보존하고, 핵심 상품을 query로 만든다. 원문에 없는 가격·유형·효능 조건을 만들지 않는다.
2. 최신 전체 데이터는 profile=test_items2를 사용한다. 카테고리를 확정하기 어렵다면 생략한다. beauty 기본값을 모든 요청에 강제하지 않는다.
3. 이 API를 사용하도록 Brain의 실제 local/remote 검색 경로를 확인한다. 환경변수 이름은 해당 Brain 구현의 계약을 따른다.
4. 반환된 product_id와 출처로 Commerce 원본을 연결한다. ranking·Commerce·근거 출처를 유지하고 무관한 상품을 가격/리뷰 점수로 다시 올리지 않는다.
5. REAL과 SYNTHETIC, 상품 근거와 리뷰 근거, 상품 단위 리뷰와 옵션 단위 리뷰를 구분한다. REAL이나 variant_id=null인 리뷰를 일괄 거부하지 않는다.
6. unverified_conditions의 조건을 "검증 완료"로 표현하지 않는다. 근거가 없는 통기성·효능·성별 적합성도 추정하지 않는다.
7. 최종 LLM 설명 실패는 검색 실패와 구분한다. 후보가 있으면 "상품 후보는 찾았지만 추천 이유와 리뷰 설명을 생성하지 못했어요"처럼 상태를 정확히 표시한다.

A-7. 현재 구현 범위와 제한

현재 검색은 상품 정체성 검사 → Commerce 가격/재고 확인 → BM25+벡터 후보 회수 → 상품 단위 RRF → 관련성 재검사 → 근거 선택 순서다. 원문 또는 query에 리뷰/후기/사용평 등이 있으면 리뷰 전용 검색 채널을 추가한다. 벡터 유사도만으로 상품군을 확정하지 않는다.

- 상품 유형 추론은 등록된 분류/별칭 기반이며 모든 상품군·복합명·오탈자·사용 목적을 이해하는 범용 분류기가 아니다.
- 전체 정규화 제목 일치 경로와 어휘 일치 우선순위는 있으나, 핵심 상품명·모델번호의 독립 후보 회수 및 최종 Brain 순위 보존 개선 요청이 모두 구현된 것은 아니다.
- required_conditions/preferences의 임의 조건은 현재 실제 필터·개인화 점수로 검증 적용되지 않는다.
- 검색된 근거는 원본 리뷰 전체가 아니며, 해당 상품의 모든 리뷰가 응답에 실리는 구조도 아니다.
- API 응답 순위와 외부 화면의 최종 순위는 Brain의 처리에 따라 달라질 수 있다.
- 기존 색인은 재사용했다. 배포된 청커 v3 코드가 기존 114,106개 청크를 v3로 바꿨다는 뜻은 아니다.
- 검색 요청만으로 카탈로그에 없는 상품을 생성하거나 구매 가능 상태를 만들지 않는다.

A-8. 기존 목록 API 및 자동 문서

경로	현재 범위
GET /v1/catalog/test-items?limit=50&cursor=0	이전 all_products/test_items 뷰티 목록에 고정
GET /v1/catalog/test-items/{product_id}/reviews	이전 test_items 리뷰 복원 응답. 리뷰 ID를 별도 해시로 반환
GET /docs	FastAPI 자동 문서
GET /openapi.json	실행 API 경로 정보

기존 목록의 limit은 1~50, cursor는 숫자 offset 0~10,000이다. 응답은 total, items, next_cursor, has_more 등을 포함한다. 이 경로에 profile=test_items2를 붙여도 새 데이터 목록으로 전환되지 않는다. test_items2 전용 전체 목록·리뷰 조회 API는 현재 제공하지 않는다.

검색 요청은 API 내부에서 JSON을 직접 검증하므로 자동 OpenAPI 문서에 검색 Body 모델이 충분히 나타나지 않을 수 있다. 제공한 rag-search.request.schema.json과 이 문서의 추가 검증 규칙, Postman collection을 함께 사용한다. JSON Schema만으로 공백·배열 요소 길이·가격 상하한 관계 같은 사용자 정의 검증이 모두 표현되지는 않는다.

부록 B. SQL·DDL·버전 자료 사용법

sql/01_toss_inventory.sql과 sql/02_test_items2_catalog.sql, sql/05_toss_linkage_checks.sql은 toss 연결에서 실행한다. sql/03_rag_active_profile.sql, sql/04_rag_product_chunks.sql, sql/06_rag_model_baseline.sql은 toss_rag 연결에서 실행한다. 모두 BEGIN READ ONLY와 ROLLBACK을 사용한다. 예시 상품 ID는 실제 존재하는 니플밴드 ID이며 운영자가 바꿔 사용할 수 있다.

DDL 파일에는 CREATE TABLE/INDEX 등 변경 명령이 들어 있다. 읽기 전용 확인 파일과 용도가 다르므로 현재 운영 DB에서 일괄 실행하지 않는다. 별도 승인된 복원/마이그레이션 환경에서 버전·확장·기존 객체를 확인한 뒤 사용한다.

requirements-runtime-snapshot.txt는 설치된 패키지의 Name==Version 목록이다. 새 아키텍처/새 Python에서 모두 설치된다는 보장이 아니며 이미지 재현 시 ARM64·Python·시스템 의존성과 함께 확인한다. supported-models JSON은 FastEmbed 등록 목록, model-cache manifest는 현재 파일 관측값이다.

현재 데이터 구조의 정본은 PostgreSQL, 실행 동작의 정본은 컨테이너와 해당 소스 커밋이다. 관측 JSON과 문서는 인수 시점 스냅샷이므로 모델·색인·데이터·권한·경로를 바꿀 때 갱신한다.

공식 기술 참조

- FastEmbed 모델/실행 개념: https://github.com/qdrant/fastembed
- pgvector 벡터 타입/거리/인덱스: https://github.com/pgvector/pgvector
- 실제 구현: https://github.com/hongbinwee/codyssey-final-toss-backend/tree/24a1051fff84bb4cf5441e90824e9e9be20e18ec/src/toss_ads/rag

현재 런타임과 실제 DB 확인을 우선하며, 최신 공식 문서의 기능이 설치된 버전과 동일하다고 가정하지 않는다.